☁️ Sesi 5: Integrasi Supabase

🎯 Overview Sesi

Durasi: 90 menit (1.5 jam) | **Branch**: `11-supabase-setup` → `12-supabase-crud` → `13-cloud-sync` | **Tujuan**: Migrate dari SharedPreferences ke Supabase database untuk cloud storage

---

📋 Agenda Sesi

Bagian 1: Supabase Setup (30 menit)

- ☑ Create Supabase account & project
- ☑ Create tasks table dengan SQL
- ☑ Disable RLS untuk kesederhanaan
- ☑ Install supabase_flutter package
- ☑ Initialize Supabase di app

Bagian 2: Replace Storage dengan Supabase (35 menit)

- ☑ Create SupabaseService class
- ☑ Implement CRUD methods dengan Supabase
- ☑ Update Controller untuk pakai SupabaseService
- ☑ Handle async operations

Bagian 3: Testing & Polish (25 menit)

- ☑ Test all CRUD operations
- ☑ Handle errors & edge cases
- ☑ Loading states & feedback
- ☑ Final testing & demo

🚀 Bagian 1: Supabase Setup (30 menit)

1.1 What is Supabase?

Supabase adalah Backend-as-a-Service (BaaS) - backend siap pakai berbasis PostgreSQL.

Analogi: - **SharedPreferences** = Lemari di rumah (hanya bisa akses dari rumah sendiri) - **Supabase** = Cloud storage (bisa akses dari mana saja, device mana saja)

Keuntungan Supabase: - ✅ **Cloud storage**: Data tersimpan di cloud, tidak hilang meski uninstall app - ✅ **Multi-device sync**: Data sama di semua device - ✅ **Real database**: PostgreSQL - powerful, scalable - ✅ **Free tier**: Generous free plan untuk belajar - ✅ **Easy to use**: Simple API, mirip seperti Firebase

Kapan pakai Supabase? - ✅ Data perlu sync antar devices - ✅ Data perlu backup di cloud - ✅ Multiple users - ✅ Data relational (ada relasi antar table)

1.2 Create Supabase Account & Project

Langkah 1: Sign Up

1. Buka https://supabase.com
2. Klik **Start your project**
3. Sign up dengan GitHub account (recommended) atau email
4. Verify email jika pakai email signup

Langkah 2: Create New Project

1. Klik **New Project**
2. Pilih organization (atau buat baru)
3. Isi form:
   - **Name**: `todo-app-workshop` (atau nama bebas)
   - **Database Password**: Buat password kuat (simpan baik-baik!)
   - **Region**: Pilih yang terdekat (e.g., `Southeast Asia (Singapore)`)
   - **Pricing Plan**: **Free** (sudah cukup untuk workshop)
4. Klik **Create new project**
5. Tunggu ~2 menit project selesai di-setup

1.3 Create Tasks Table

Langkah 1: Open SQL Editor

1. Di Supabase Dashboard, klik **SQL Editor** di sidebar
2. Klik **New query**

Langkah 2: Create Table dengan SQL

Copy dan paste SQL ini, lalu klik **Run**:

```sql
-- Create tasks table
CREATE TABLE tasks (
  id TEXT PRIMARY KEY,
  title TEXT NOT NULL,
  description TEXT DEFAULT '',
  is_completed BOOLEAN DEFAULT false,
  created_at TIMESTAMP WITH TIME ZONE NOT NULL,
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

```sql
-- Create index untuk performa
CREATE INDEX idx_tasks_created_at ON tasks(created_at DESC);
CREATE INDEX idx_tasks_is_completed ON tasks(is_completed);

-- Add trigger untuk auto-update updated_at
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
  NEW.updated_at = NOW();
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER update_tasks_updated_at
  BEFORE UPDATE ON tasks
  FOR EACH ROW
  EXECUTE FUNCTION update_updated_at_column();

-- Insert sample data untuk testing
INSERT INTO tasks (id, title, description, is_completed, created_at)
VALUES
  ('sample-1', 'Welcome to Supabase!', 'This is a sample task from
       cloud database', false, NOW()),
  ('sample-2', 'Test cloud sync', 'Try adding, editing, and deleting
       tasks', false, NOW());
```

💡 **Penjelasan:**

- **id TEXT PRIMARY KEY**: ID sebagai string (sama seperti di app)
- **title TEXT NOT NULL**: Title wajib diisi
- **description TEXT DEFAULT ''**: Description optional, default empty
- **is_completed BOOLEAN**: Status completed true/false
- **created_at**: Timestamp kapan dibuat
- **updated_at**: Auto-update setiap kali record di-update (via trigger)

Langkah 3: Verify Table Created

1. Klik **Table Editor** di sidebar
2. Lihat table `tasks` dengan 2 sample rows
3. Coba klik row untuk lihat detail

1.4 Disable RLS (Row Level Security)

Untuk workshop ini, kita disable RLS agar lebih sederhana (tidak perlu authentication).

⚠️ **Note**: Di production app, WAJIB pakai RLS + Auth untuk security!

Langkah 1: Open SQL Editor

Jalankan SQL ini:

```sql
-- Disable RLS untuk tasks table
ALTER TABLE tasks DISABLE ROW LEVEL SECURITY;
```

Sekarang semua client bisa akses data tanpa authentication.

1.5 Get API Keys

Langkah 1: Open Project Settings

1. Klik **Settings** icon (gear) di sidebar bawah
2. Klik **API**

Langkah 2: Copy Keys

Copy 2 values ini (simpan di notepad): - **Project URL**: `https://xxxxx.supabase.co` - **anon/public key**: `eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9....` (long string)

1.6 Install Supabase Flutter Package

Langkah 1: Add Dependencies

Edit file: `pubspec.yaml`

```yaml
dependencies:
  flutter:
    sdk: flutter
  nylo_framework: ^6.9.1
  shared_preferences: ^2.5.3
  supabase_flutter: ^2.9.1  # Tambahkan ini
```

Langkah 2: Install Package

```
flutter pub get
```

1.7 Initialize Supabase di App

Langkah 1: Create .env File

Create file: `.env` (di root project, sejajar dengan `pubspec.yaml`)

```
# Supabase Configuration
SUPABASE_URL=https://your-project.supabase.co
SUPABASE_ANON_KEY=your-anon-key-here
```

⚠️ **PENTING:** - Ganti `your-project.supabase.co` dengan Project URL Anda - Ganti `your-anon-key-here` dengan anon key Anda

Langkah 2: Add .env ke .gitignore

Edit file: `.gitignore`

Tambahkan di baris paling bawah:

```
# Environment variables
.env
```

Ini penting agar API keys tidak ter-commit ke git!

Langkah 3: Load .env di pubspec.yaml

Edit file: `pubspec.yaml`

Di bagian `flutter:`, tambahkan:

```yaml
flutter:
  uses-material-design: true

  # Add .env file
  assets:
    - .env
```

Langkah 4: Initialize Supabase

Edit file: `lib/main.dart`

```dart
import 'package:flutter/material.dart';
import 'package:nylo_framework/nylo_framework.dart';
import 'package:supabase_flutter/supabase_flutter.dart';
import 'package:flutter/services.dart';
import 'bootstrap/boot.dart';

/// Main entry point for the application.
void main() async {
  WidgetsFlutterBinding.ensureInitialized();

  // Load .env file
  String envContent = await rootBundle.loadString('.env');
  Map<String, String> envVars = {};

  for (String line in envContent.split('\n')) {
    line = line.trim();
    // Skip comments and empty lines
    if (line.isEmpty || line.startsWith('#')) continue;

    // Parse KEY=VALUE
    int separatorIndex = line.indexOf('=');
    if (separatorIndex != -1) {
      String key = line.substring(0, separatorIndex).trim();
      String value = line.substring(separatorIndex + 1).trim();
      envVars[key] = value;
    }
  }

  // Initialize Supabase
  try {
    await Supabase.initialize(
      url: envVars['SUPABASE_URL'] ?? '',
      anonKey: envVars['SUPABASE_ANON_KEY'] ?? '',
```

```
    );
    print('✅ Supabase initialized successfully');
  } catch (e) {
    print('⚠️ Supabase initialization error: $e');
    print('⚠️ Please configure .env file with your Supabase
        credentials');
  }

  // Initialize Nylo
  await Nylo.init(
    setup: Boot.nylo,
    setupFinished: Boot.finished,
  );
}
```

💡 **Penjelasan:** - `WidgetsFlutterBinding.ensureInitialized()`: Required untuk async operations di main - Manual parsing `.env` menggunakan `rootBundle.loadString()` - tidak perlu package tambahan - Parse setiap line format `KEY=VALUE` - Skip comment lines (dimulai dengan `#`) dan empty lines - `Supabase.initialize()`: Initialize Supabase client dengan credentials - Wrapped dengan try-catch untuk handle error jika .env belum dikonfigurasi - `Nylo.init()`: Initialize Nylo framework setelah Supabase

Langkah 5: Test Initialization

Hot restart app (Shift + R di terminal atau stop & run ulang)

Cek console output:

✅ `Supabase initialized successfully`

Jika ada error ⚠️ `Supabase initialization error`, berarti: - File `.env` belum ada atau belum dikonfigurasi - URL atau anon key salah - File `.env` belum ditambahkan ke `pubspec.yaml` di bagian assets

Jika tidak ada error, Supabase berhasil di-initialize! 🎉

🔄 Bagian 2: Replace Storage dengan Supabase (35 menit)

2.1 Create SupabaseService

Create file: `lib/app/services/supabase_service.dart`

```
import 'package:supabase_flutter/supabase_flutter.dart';
import '/app/models/task.dart';

class SupabaseService {
  // Get Supabase client
  final SupabaseClient supabase = Supabase.instance.client;

  // Table name
```

```dart
static const String tableName = 'tasks';

/// Fetch all tasks from Supabase
Future<List<Task>> getTasks() async {
  try {
    final response = await supabase
        .from(tableName)
        .select()
        .order('created_at', ascending: false);

    // Convert response to List<Task>
    List<Task> tasks = (response as List)
        .map((json) => Task.fromSupabaseJson(json))
        .toList();

    print('✅ Fetched ${tasks.length} tasks from Supabase');
    return tasks;
  } catch (e) {
    print('❌ Error fetching tasks: $e');
    rethrow;
  }
}

/// Insert new task to Supabase
Future<Task> createTask(Task task) async {
  try {
    final response = await supabase
        .from(tableName)
        .insert(task.toSupabaseJson())
        .select()
        .single();

    Task createdTask = Task.fromSupabaseJson(response);
    print('✅ Created task in Supabase: ${createdTask.id}');
    return createdTask;
  } catch (e) {
    print('❌ Error creating task: $e');
    rethrow;
  }
}

/// Update existing task in Supabase
Future<Task> updateTask(Task task) async {
  try {
    final response = await supabase
        .from(tableName)
        .update(task.toSupabaseJson())
        .eq('id', task.id)
        .select()
        .single();

    Task updatedTask = Task.fromSupabaseJson(response);
    print('✅ Updated task in Supabase: ${updatedTask.id}');
    return updatedTask;
  } catch (e) {
```

```
      print('❌ Error updating task: $e');
      rethrow;
    }
  }

  /// Delete task from Supabase
  Future<void> deleteTask(String taskId) async {
    try {
      await supabase
          .from(tableName)
          .delete()
          .eq('id', taskId);

      print('✅ Deleted task from Supabase: $taskId');
    } catch (e) {
      print('❌ Error deleting task: $e');
      rethrow;
    }
  }

  /// Delete all tasks
  Future<void> deleteAllTasks() async {
    try {
      // Get all task IDs first
      final tasks = await getTasks();

      // Delete one by one (untuk table tanpa RLS bisa delete all
         sekaligus)
      for (var task in tasks) {
        await deleteTask(task.id);
      }

      print('✅ Deleted all tasks from Supabase');
    } catch (e) {
      print('❌ Error deleting all tasks: $e');
      rethrow;
    }
  }
}
```

2.2 Update Task Model untuk Supabase

Edit file: `lib/app/models/task.dart`

Tambahkan methods untuk convert dari/ke Supabase format:

```
class Task {
  // Properties - tetap sama
  String id;
  String title;
  String description;
  bool isCompleted;
  DateTime createdAt;
```

```dart
// Constructor - tetap sama
Task({
  required this.id,
  required this.title,
  this.description = '',
  this.isCompleted = false,
  required this.createdAt,
});

// Existing fromJson untuk SharedPreferences
factory Task.fromJson(Map<String, dynamic> json) {
  return Task(
    id: json['id'] as String,
    title: json['title'] as String,
    description: json['description'] as String? ?? '',
    isCompleted: json['isCompleted'] as bool? ?? false,
    createdAt: DateTime.parse(json['createdAt'] as String),
  );
}

// Existing toJson untuk SharedPreferences
Map<String, dynamic> toJson() {
  return {
    'id': id,
    'title': title,
    'description': description,
    'isCompleted': isCompleted,
    'createdAt': createdAt.toIso8601String(),
  };
}

// NEW: fromSupabaseJson - convert dari Supabase response
factory Task.fromSupabaseJson(Map<String, dynamic> json) {
  return Task(
    id: json['id'] as String,
    title: json['title'] as String,
    description: json['description'] as String? ?? '',
    isCompleted: json['is_completed'] as bool? ?? false,  // Note:
      snake_case dari database
    createdAt: DateTime.parse(json['created_at'] as String),  //
      Note: snake_case
  );
}

// NEW: toSupabaseJson - convert ke format Supabase
Map<String, dynamic> toSupabaseJson() {
  return {
    'id': id,
    'title': title,
    'description': description,
    'is_completed': isCompleted,  // Convert ke snake_case untuk
      database
    'created_at': createdAt.toIso8601String(),
  };
}
```

```
  // copyWith - tetap sama
  Task copyWith({
    String? id,
    String? title,
    String? description,
    bool? isCompleted,
    DateTime? createdAt,
  }) {
    return Task(
      id: id ?? this.id,
      title: title ?? this.title,
      description: description ?? this.description,
      isCompleted: isCompleted ?? this.isCompleted,
      createdAt: createdAt ?? this.createdAt,
    );
  }

  @override
  String toString() {
    return 'Task(id: $id, title: $title, isCompleted: $isCompleted)';
  }
}
```

💡 **Perbedaan Key:** - **SharedPreferences**: `isCompleted`, `createdAt` (camelCase) - **Supabase Database**: `is_completed`, `created_at` (snake_case)

Kita buat 2 set methods untuk handle keduanya!

2.3 Update Controller untuk Pakai Supabase

Edit file: `lib/app/controllers/todo_home_controller.dart`

```
import 'package:flutter/material.dart';
import 'package:nylo_framework/nylo_framework.dart';
import '/app/models/task.dart';
import '/app/services/supabase_service.dart';

class TodoHomeController extends NyController {
  // State
  List<Task> tasks = [];
  bool isLoading = false;
  String? errorMessage;

  // Supabase service (ganti dari StorageService)
  final SupabaseService supabase = SupabaseService();

  @override
  construct(BuildContext context) async {
    super.construct(context);

    // Load tasks from Supabase
    await loadTasksFromSupabase();
  }
```

```dart
/// Load tasks from Supabase
Future<void> loadTasksFromSupabase() async {
  isLoading = true;
  errorMessage = null;
  setState(setState: () {});

  try {
    // Fetch dari Supabase
    List<Task> loadedTasks = await supabase.getTasks();

    tasks = loadedTasks;
    isLoading = false;
    setState(setState: () {});

    print('✅ Loaded ${tasks.length} tasks from Supabase');
  } catch (e) {
    print('❌ Error loading tasks: $e');

    isLoading = false;
    errorMessage = 'Failed to load tasks: ${e.toString()}';
    setState(setState: () {});
  }
}

/// Refresh tasks (pull-to-refresh)
Future<void> refresh() async {
  await loadTasksFromSupabase();
}

// Helper getters
int get totalTasks => tasks.length;
int get completedTasks => tasks.where((t) => t.isCompleted).length;
int get pendingTasks => tasks.where((t) => !t.isCompleted).length;

/// Add new task to Supabase
Future<void> addTask({
  required String title,
  String description = '',
}) async {
  try {
    // Create task object
    Task newTask = Task(
      id: DateTime.now().millisecondsSinceEpoch.toString(),
      title: title,
      description: description,
      isCompleted: false,
      createdAt: DateTime.now(),
    );

    // Insert ke Supabase
    Task createdTask = await supabase.createTask(newTask);

    // Add to local list
    tasks.insert(0, createdTask);
```

```
    setState(setState: () {});

    print('✅ Task added to Supabase: $createdTask');
  } catch (e) {
    print('❌ Error adding task: $e');
    rethrow;
  }
}

/// Update task in Supabase
Future<void> updateTask({
  required String id,
  String? title,
  String? description,
  bool? isCompleted,
}) async {
  try {
    final taskIndex = tasks.indexWhere((t) => t.id == id);

    if (taskIndex == -1) {
      throw Exception('Task not found');
    }

    // Create updated task
    Task updatedTask = tasks[taskIndex].copyWith(
      title: title,
      description: description,
      isCompleted: isCompleted,
    );

    // Update di Supabase
    Task savedTask = await supabase.updateTask(updatedTask);

    // Update local list
    tasks[taskIndex] = savedTask;

    setState(setState: () {});

    print('✅ Task updated: $savedTask');
  } catch (e) {
    print('❌ Error updating task: $e');
    rethrow;
  }
}

/// Toggle complete status
Future<void> toggleComplete(String id) async {
  final taskIndex = tasks.indexWhere((t) => t.id == id);

  if (taskIndex != -1) {
    await updateTask(
      id: id,
      isCompleted: !tasks[taskIndex].isCompleted,
    );
```

```
    }
  }

  /// Delete task from Supabase
  Future<void> deleteTask(String id) async {
    try {
      // Delete dari Supabase
      await supabase.deleteTask(id);

      // Remove from local list
      tasks.removeWhere((t) => t.id == id);

      setState(setState: () {});

      print('✅ Task deleted: $id');
    } catch (e) {
      print('❌ Error deleting task: $e');
      rethrow;
    }
  }

  /// Clear all tasks
  Future<void> clearAllTasks() async {
    try {
      await supabase.deleteAllTasks();

      tasks.clear();

      setState(setState: () {});

      print('✅ All tasks cleared');
    } catch (e) {
      print('❌ Error clearing tasks: $e');
      rethrow;
    }
  }

  // Filter methods
  List<Task> getTasksByStatus(bool isCompleted) {
    return tasks.where((t) => t.isCompleted == isCompleted).toList();
  }

  List<Task> get pendingTasksList => getTasksByStatus(false);
  List<Task> get completedTasksList => getTasksByStatus(true);
}
```

💡 **Perubahan penting dari docs sebelumnya:** - `extends Controller` → `extends NyController` (API Nylo terbaru) - `construct()` method dibuat `async` untuk await `loadTasksFromSupabase()` - `setState(() {})` → `setState(setState: () {})` (named parameter) - Removed `showToastNotification` calls dari controller (lebih baik di UI layer) - `isLoading` state di-manage langsung tanpa wrap setState - Error handling lebih sederhana dengan rethrow

---

2.4 Direct Supabase Access dari Pages

Untuk page yang tidak memiliki controller association (seperti `AddTaskPage` dan `DetailTaskPage`), kita bisa langsung akses Supabase client tanpa melalui controller.

Add Task Page dengan Direct Supabase

Edit file: `lib/resources/pages/add_task_page.dart`

Pada method `_handleSave()`, tambahkan direct Supabase insert:

```dart
Future<void> _handleSave() async {
  String title = titleController.text.trim();
  String description = descriptionController.text.trim();

  if (title.isEmpty) {
    showToastNotification(
      context,
      title: "Error",
      description: "Task title tidak boleh kosong!",
      icon: Icons.error,
      style: ToastNotificationStyleType.danger,
    );
    return;
  }

  setState(() {
    isSaving = true;
  });

  try {
    // Create new task directly with Supabase
    // Generate ID using timestamp (same as controller)
    final taskId = DateTime.now().millisecondsSinceEpoch.toString();

    await Supabase.instance.client.from('tasks').insert({
      'id': taskId,
      'title': title,
      'description': description,
      'is_completed': false,
      'created_at': DateTime.now().toIso8601String(),
    });

    showToastNotification(
      context,
      title: "Success",
      description: "Task berhasil ditambahkan dan disimpan!",
      icon: Icons.check_circle,
      style: ToastNotificationStyleType.success,
    );

    // Wait a bit before popping
    await Future.delayed(Duration(milliseconds: 500));

    if (mounted) {
```

```
      Navigator.pop(context, true); // Return true to indicate task
        was added
    }
  } catch (e) {
    print('Error saving task: $e');

    showToastNotification(
      context,
      title: "Error",
      description: "Gagal menyimpan task!",
      icon: Icons.error,
      style: ToastNotificationStyleType.danger,
    );

    setState(() {
      isSaving = false;
    });
  }
}
```

💡 **Poin penting:** - Generate ID menggunakan timestamp (sama seperti di controller) - Direct insert ke Supabase tanpa melalui service/controller - Return `true` saat Navigator.pop untuk signal bahwa task ditambahkan - Parent page akan refresh saat menerima return value `true`

Detail Task Page dengan Edit Feature

`DetailTaskPage` juga menggunakan direct Supabase access untuk update dan delete:

```
// Update task status
Future<void> _toggleTaskStatus() async {
  if (task == null) return;

  setState(() {
    isProcessing = true;
  });

  try {
    // Update task status directly with Supabase
    final newStatus = !task!.isCompleted;
    await Supabase.instance.client
        .from('tasks')
        .update({'is_completed': newStatus})
        .eq('id', task!.id);

    setState(() {
      task = task!.copyWith(isCompleted: newStatus);
      isProcessing = false;
      hasChanges = true; // Mark that task was modified
    });

    showToastNotification(
      context,
      title: "Success",
      description: "Task status updated and saved to cloud!",
```

```
      icon: Icons.check_circle,
      style: ToastNotificationStyleType.success,
    );
  } catch (e) {
    // Error handling...
  }
}

// Save edited task
Future<void> _saveChanges() async {
  if (task == null) return;

  final newTitle = titleController.text.trim();
  final newDescription = descriptionController.text.trim();

  if (newTitle.isEmpty) {
    showToastNotification(
      context,
      title: "Error",
      description: "Task title cannot be empty!",
      icon: Icons.error,
      style: ToastNotificationStyleType.danger,
    );
    return;
  }

  setState(() {
    isProcessing = true;
  });

  try {
    // Update task in Supabase
    await Supabase.instance.client.from('tasks').update({
      'title': newTitle,
      'description': newDescription,
    }).eq('id', task!.id);

    setState(() {
      task = task!.copyWith(
        title: newTitle,
        description: newDescription,
      );
      isProcessing = false;
      isEditMode = false;
      hasChanges = true;
    });

    showToastNotification(
      context,
      title: "Success",
      description: "Task updated successfully!",
      icon: Icons.check_circle,
      style: ToastNotificationStyleType.success,
    );
  } catch (e) {
```

```
    // Error handling...
  }
}

// Delete task
Future<void> _deleteTask() async {
  if (task == null) return;

  setState(() {
    isProcessing = true;
  });

  try {
    // Delete task directly with Supabase
    await Supabase.instance.client
        .from('tasks')
        .delete()
        .eq('id', task!.id);

    showToastNotification(
      context,
      title: "Success",
      description: "Task deleted and saved to cloud!",
      icon: Icons.delete,
      style: ToastNotificationStyleType.success,
    );

    await Future.delayed(Duration(milliseconds: 500));

    if (mounted) {
      Navigator.pop(context, true); // Return true to indicate
        modification
    }
  } catch (e) {
    // Error handling...
  }
}
```

💡 **Pattern untuk change tracking:** - Tambahkan flag `hasChanges` dan `isEditMode` di state - Set `hasChanges = true` setiap kali task dimodifikasi (toggle, edit, delete) - Return `hasChanges` saat back button pressed menggunakan PopScope - Parent page akan refresh jika menerima `true`

```
return PopScope(
  canPop: false,
  onPopInvokedWithResult: (didPop, result) async {
    if (!didPop) {
      // Handle back button - return hasChanges flag
      Navigator.pop(context, hasChanges);
    }
  },
  child: Scaffold(
    appBar: AppBar(
      leading: IconButton(
        icon: Icon(Icons.arrow_back),
```

```
        onPressed: () {
          Navigator.pop(context, hasChanges);
        },
      ),
      // ...
    ),
    // ...
  ),
);
```

2.5 Update Home Pages untuk Auto-Refresh

Update navigasi di home pages untuk listen return value dan refresh:

Edit file: `lib/resources/pages/home_page.dart` dan `todo_home_page.dart`

```
// Add Task navigation
floatingActionButton: FloatingActionButton(
  onPressed: () async {
    final result = await Navigator.pushNamed(context, '/add-task');
    if (result == true) {
      // Task was added, refresh the list
      widget.controller.refresh();
    }
  },
  child: Icon(Icons.add),
),

// Detail Task navigation (in ListView item)
onTap: () async {
  final result = await Navigator.pushNamed(
    context,
    '/detail-task',
    arguments: task.toJson(),
  );
  if (result == true) {
    // Task was modified or deleted, refresh the list
    widget.controller.refresh();
  }
},
```

💡 **Keuntungan pattern ini:** - UI selalu up-to-date dengan cloud data - User tidak perlu manual refresh - Clean separation: pages handle CRUD, controller manage state - Lebih flexible untuk pages tanpa controller association

---

2.6 Add Loading Indicator to UI

Edit file: `lib/resources/pages/todo_home_page.dart`

Update `_buildTaskList()` untuk show loading:

```
Widget _buildTaskList() {
  final controller = widget.controller;
```

```dart
// Show loading indicator
if (controller.isLoading && controller.tasks.isEmpty) {
  return Center(
    child: Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        CircularProgressIndicator(),
        SizedBox(height: 16),
        Text('Loading tasks from cloud...'),
      ],
    ),
  );
}

// Show error message
if (controller.errorMessage != null) {
  return Center(
    child: Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        Icon(Icons.cloud_off, size: 64, color: Colors.red),
        SizedBox(height: 16),
        Text(
          'Failed to load tasks',
          style: TextStyle(fontSize: 18, fontWeight:
     FontWeight.bold),
        ),
        SizedBox(height: 8),
        Text(
          controller.errorMessage!,
          textAlign: TextAlign.center,
          style: TextStyle(color: Colors.grey),
        ),
        SizedBox(height: 16),
        ElevatedButton.icon(
          onPressed: () {
            controller.refresh();
          },
          icon: Icon(Icons.refresh),
          label: Text('Retry'),
        ),
      ],
    ),
  );
}

List<Task> tasks = controller.tasks;

// Empty state
if (tasks.isEmpty) {
  return Center(
    child: Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
```

```
            Icon(Icons.cloud_done, size: 80, color:
          Colors.grey.shade300),
            SizedBox(height: 16),
            Text(
              'No tasks yet!',
              style: TextStyle(
                fontSize: 20,
                color: Colors.grey.shade600,
                fontWeight: FontWeight.bold,
              ),
            ),
            SizedBox(height: 8),
            Text(
              'Tap the + button to add your first task',
              style: TextStyle(
                fontSize: 14,
                color: Colors.grey.shade500,
              ),
            ),
          ],
        ),
      );
    }

    return ListView.builder(
      padding: EdgeInsets.all(16),
      itemCount: tasks.length,
      itemBuilder: (context, index) {
        final task = tasks[index];
        return _buildTaskCard(task);
      },
    );
  }
```

✅ Bagian 3: Testing & Polish (25 menit)

3.1 Complete Testing Checklist

Test semua operasi CRUD dengan Supabase:

Create: - [ ] Add task baru - [ ] Task muncul di app - [ ] Check di Supabase Table Editor - task ada di database - [ ] Close app, open lagi - task masih ada

Read: - [ ] Open app - tasks load dari Supabase - [ ] Loading indicator muncul saat fetch - [ ] Empty state jika belum ada tasks

Update: - [ ] Toggle complete task - [ ] Status update di app - [ ] Check di Supabase - is_completed berubah - [ ] Refresh app - status tetap

Delete: - [ ] Delete task - [ ] Task hilang dari app - [ ] Check di Supabase - task hilang dari database

Multi-Device (Bonus): - [ ] Install app di 2 devices (atau 1 device + browser Supabase) - [ ] Add task di device 1 - [ ] Refresh/restart app di device 2 - [ ] Task muncul di device 2! 🎉

3.2 Add Pull-to-Refresh

Edit file: `lib/resources/pages/todo_home_page.dart`

Wrap ListView dengan RefreshIndicator:

```dart
@override
Widget view(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text('My Todo List'),
      backgroundColor: Colors.blue,
      foregroundColor: Colors.white,
      actions: [
        // Refresh button
        IconButton(
          icon: Icon(Icons.refresh),
          onPressed: () {
            widget.controller.refresh();
          },
        ),
        PopupMenuButton<String>(
          onSelected: (value) {
            if (value == 'clear') {
              _showClearConfirmation();
            }
          },
          itemBuilder: (context) => [
            PopupMenuItem(
              value: 'clear',
              child: Row(
                children: [
                  Icon(Icons.delete_sweep, color: Colors.red),
                  SizedBox(width: 8),
                  Text('Clear All Tasks'),
                ],
              ),
            ),
          ],
        ),
      ],
    ),
    body: Column(
      children: [
        _buildStatsCard(),
        Expanded(
          child: RefreshIndicator(
            onRefresh: () => widget.controller.refresh(),
            child: _buildTaskList(),
```

```
          ),
        ),
      ],
    ),
    floatingActionButton: FloatingActionButton(
      onPressed: () {
        routeTo('/add-task');
      },
      child: Icon(Icons.add),
    ),
  );
}
```

Now users can pull down to refresh! 🔄

3.3 Error Handling Best Practices

Pastikan semua async operations di-wrap dengan try-catch:

```
// Good ✅
try {
  await supabase.createTask(newTask);
  // Success handling
} catch (e) {
  // Error handling
  showToastNotification(
    context,
    title: 'Error',
    description: 'Something went wrong',
    style: ToastNotificationStyleType.DANGER,
  );
}

// Bad ❌
await supabase.createTask(newTask); // Bisa crash kalau error!
```

🎉 Congratulations!

Aplikasi ToDo List Anda sekarang: ✅ Pakai Model class (type-safe) ✅ Data persist dengan Supabase (cloud storage) ✅ CRUD operations lengkap ✅ Multi-device sync ready ✅ Error handling proper ✅ Loading states & feedback

---

🚀 Next Steps (After Workshop)

Improvements:

1. **Authentication**: Add user login dengan Supabase Auth

2. **RLS**: Enable Row Level Security untuk data privacy
3. **Offline Mode**: Cache data dengan Hive untuk offline-first
4. **Real-time**: Subscribe ke Supabase Realtime untuk live updates
5. **Search & Filter**: Tambah fitur search dan filter by status
6. **Categories**: Buat multiple task lists/categories
7. **Due Dates**: Tambah due date & reminders
8. **UI Polish**: Better animations, themes, dark mode

🐛 Common Issues & Solutions

Issue 0: "type 'Null' is not a subtype of type 'String'" di theme.dart

Symptom: App crash saat startup dengan error di `appThemes` configuration **Solution**:

Theme configuration di Nylo menggunakan environment variables untuk theme IDs. Jika `.env` tidak ada atau tidak loaded, `getEnv()` return `null` yang menyebabkan error.

Fix di `lib/config/theme.dart`:

```
// SEBELUM (Error jika .env belum ada)
final List<BaseThemeConfig<ColorStyles>> appThemes = [
  BaseThemeConfig<ColorStyles>(
    id: getEnv('LIGHT_THEME_ID'),  // ❌ Bisa return null
    description: "Light theme",
    theme: lightTheme,
    colors: LightThemeColors(),
  ),
  // ...
];

// SESUDAH (Always works)
final List<BaseThemeConfig<ColorStyles>> appThemes = [
  BaseThemeConfig<ColorStyles>(
    id: 'light_theme',  // ✅ Hardcoded string
    description: "Light theme",
    theme: lightTheme,
    colors: LightThemeColors(),
  ),
  BaseThemeConfig<ColorStyles>(
    id: 'dark_theme',
    description: "Dark theme",
    theme: darkTheme,
    colors: DarkThemeColors(),
  ),
];
```

Untuk workshop ini, lebih simple pakai hardcoded theme IDs. Di production app, bisa tetap pakai `getEnv()` dengan fallback:

```
id: getEnv('LIGHT_THEME_ID') ?? 'light_theme',
```

Issue 1: “Connection refused” atau “Failed to connect”

Symptom: Error saat fetch dari Supabase **Solution**: - Check internet connection - Verify SUPABASE_URL dan SUPABASE_ANON_KEY di .env - Check di Supabase Dashboard - project status = Active - Coba access URL di browser - harus response

Issue 2: “Invalid API key”

Symptom: 401 Unauthorized error **Solution**: - Copy ulang anon key dari Supabase Dashboard → Settings → API - Pastikan tidak ada extra spaces di .env - Restart app setelah ubah .env

Issue 3: “Table ‘tasks’ does not exist”

Symptom: Error query table not found **Solution**: - Check di Table Editor - table `tasks` ada? - Re-run SQL create table script - Verify table name sama persis (case-sensitive)

Issue 4: “Column ‘is_completed’ not found”

Symptom: Error query column not found **Solution**: - Check column names di Table Editor - Pastikan pakai snake_case: `is_completed`, `created_at` - Update `toSupabaseJson()` dan `fromSupabaseJson()` methods

Issue 5: Tasks not syncing between devices

Symptom: Add di device A, tidak muncul di device B **Solution**: - Pastikan kedua device pakai project Supabase yang sama - Call `refresh()` di device B untuk fetch latest data - Check internet di kedua devices

---

📚 Key Takeaways

✅ **Supabase:** - Backend-as-a-Service berbasis PostgreSQL - Free tier generous untuk belajar & small projects - Easy integration dengan Flutter via supabase_flutter package

✅ **Migration Path:** - Start simple: In-memory (List) - Then persist: SharedPreferences (local) - Scale up: Supabase (cloud)

✅ **Best Practices:** - Always use .env untuk credentials (jangan hardcode!) - Add .env ke .gitignore - Wrap async operations dengan try-catch - Show loading states untuk better UX - Handle errors gracefully dengan user feedback

✅ **Database Naming:** - Dart/Flutter: camelCase (`isCompleted`) - Database: snake_case (`is_completed`) - Convert di Model methods: `fromSupabaseJson()` & `toSupabaseJson()`

📖 Resources

- **Supabase Docs**: https://supabase.com/docs

- **Supabase Flutter**: https://supabase.com/docs/reference/dart/introduction
- **Flutter Dotenv**: https://pub.dev/packages/flutter_dotenv
- **PostgreSQL Tutorial**: https://www.postgresqltutorial.com/

🔗 Navigation

← Sesi 4 - Model & Data Persistence | Workshop Modules

Workshop Material - Simple ToDo List with Flutter Nylo | Dokumentasi Sesi 5 - Integrasi Supabase | Terakhir diperbarui: November 22, 2025